Geth Verkle 명령어 상세 분석

`geth verkle` 명령어는 실험적인 Verkle Tree 관리 도구로, Ethereum의 차세대 상태 트리 구조를 다루는 명령어입니다.

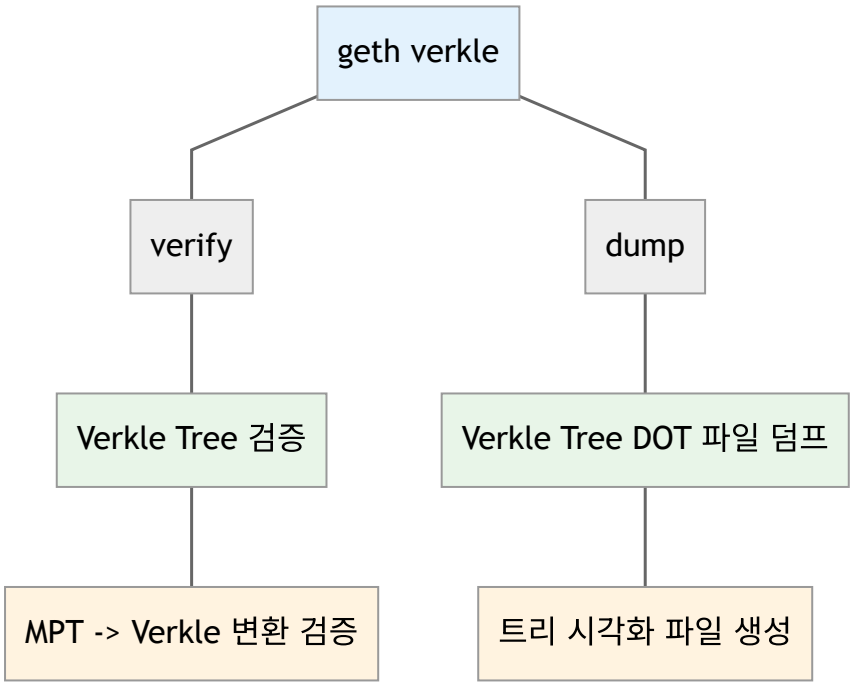
> **명령어:** `geth verkle <subcommand>`

> **진입점:** `verkleCommand` → 각 서브명령어 함수

> **소스 파일:** `cmd/geth/verkle.go`

> **카테고리:** 실험적 기능

🚀 전체 명령어 구조



📋 Verkle Tree 개요

Verkle Tree란?

Verkle Tree는 Ethereum의 차세대 상태 트리 구조로, 기존 Merkle Patricia Tree(MPT)의 한계를 극복하기 위해 개발되었습니다.

주요 특징:

- **Vector Commitments**: 더 효율적인 증명 생성
- **작은 증명 크기**: 상태 증명이 기존 대비 크게 축소
- **Stateless 클라이언트**: 전체 상태 없이도 블록 검증 가능
- **확장성**: 더 많은 트랜잭션 처리 가능

기술적 차이점:

```
Merkle Patricia Tree (현재) Verkle Tree (미래)

├── 바이너리 트리 구조 ├── 256-ary 트리 구조

├── SHA256/Keccak256 해시 ├── Pedersen 커밋먼트

├── 큰 증명 크기 (~1KB) ├── 작은 증명 크기 (~150B)

└── 상태 증명 복잡 └── 간단한 상태 증명
```

📋 명령어 정의 및 구조

메인 Verkle 명령어

```
// cmd/geth/verkle.go

verkleCommand = &cli.Command{

Name: "verkle",

Usage: "A set of experimental verkle tree management commands",

Description: "",
```

```
Subcommands: []*cli.Command{

{

Name: "verify",

Usage: "verify the conversion of a MPT into a verkle tree",

ArgsUsage: "<root>",

Action: verifyVerkle,

// ...

},

{

Name: "dump",

Usage: "Dump a verkle tree to a DOT file",

ArgsUsage: "<root> <key1> [<key 2> ...]",

Action: expandVerkle,

// ...

},

},

}
```

📋 서브명령어별 상세 분석

1. `geth verkle verify` - Verkle Tree 검증

```
{
```

```
Name: "verify",

Usage: "verify the conversion of a MPT into a verkle tree",

ArgsUsage: "<root>",

Action: verifyVerkle,

Flags: slices.Concat(utils.NetworkFlags, utils.DatabaseFlags),

Description: `

geth verkle verify <state-root>

This command takes a root commitment and attempts to rebuild the tree.

`,

}



// verifyVerkle 함수

func verifyVerkle(ctx *cli.Context) error {

// 1. 노드 스택 생성

stack, _ := makeConfigNode(ctx)

defer stack.Close()

// 2. 체인 데이터베이스 열기 (읽기 전용)

chaindb := utils.MakeChainDatabase(ctx, stack, true)

defer chaindb.Close()

// 3. 헤드 블록 로드

headBlock := rawdb.ReadHeadBlock(chaindb)

if headBlock == nil {

log.Error("Failed to load head block")
```

```
return errors.New("no head block")
}
// 4. 루트 커밋먼트 결정
var (
rootC common.Hash
err error
)
if ctx.NArg() == 1 {
// 명령행에서 지정된 루트 사용
rootC, err = parseRoot(ctx.Args().First())
if err != nil {
log.Error("Failed to resolve state root", "error", err)
return err
}
log.Info("Rebuilding the tree", "root", rootC)
} else {
// 헤드 블록의 루트 사용
rootC = headBlock.Root()
log.Info("Rebuilding the tree", "root", rootC, "number", headBlock.NumberU64())
}
// 5. 직렬화된 루트 노드 가져오기
serializedRoot, err := chaindb.Get(rootC[:])
if err != nil {
```

```
return fmt.Errorf("failed to get root from database: %v", err)
}
// 6. Verkle 노드 파싱
root, err := verkle.ParseNode(serializedRoot, 0)
if err != nil {
return fmt.Errorf("failed to parse verkle node: %v", err)
}
// 7. 트리 재구성 및 검증
if err := checkChildren(root, chaindb.Get); err != nil {
log.Error("Could not rebuild the tree from the database", "err", err)
return err
}
log.Info("Tree was rebuilt from the database")
return nil
}
```

자식 노드 검증 함수 (`checkChildren()`)

```
// checkChildren는 재귀적으로 모든 자식 노드를 검증합니다.
func checkChildren(root verkle.VerkleNode, resolver verkle.NodeResolverFn) erro
switch node := root.(type) {
case *verkle.InternalNode:
// 1. 내부 노드의 모든 자식 검증
for i, child := range node.Children() {
```

```
childC := child.Commit().Bytes()

// 2. 빈 노드 건너뛰기

if bytes.Equal(childC[:], zero[:]) {

continue

}

// 3. 데이터베이스에서 자식 노드 로드

childS, err := resolver(childC[:])

if err != nil {

return fmt.Errorf("could not find child %x in db: %w", childC, err)

}

// 4. 자식 노드 파싱

childN, err := verkle.ParseNode(childS, 0)

if err != nil {

return fmt.Errorf("decode error child %x in db: %w", child.Commitment().Bytes()

}

// 5. 재귀적으로 자식의 자식들 검증

if err := checkChildren(childN, resolver); err != nil {

return fmt.Errorf("%x%w", i, err) // 오류 경로 추가

}

}

case *verkle.LeafNode:

// 6. 리프 노드 검증: 최소 하나의 값이 0이 아닌지 확인

for i := 0; i < verkle.NodeWidth; i++ {
```

```
if len(node.Value(i)) != 0 {
return nil // 유효한 값 발견
}
}
return errors.New("both balance and nonce are 0")
case verkle.Empty:
// 7. 빈 노드는 검증할 것이 없음
default:
return fmt.Errorf("unsupported type encountered %v", root)
}
return nil
}
```

사용 예시:

```
# 헤드 블록의 Verkle Tree 검증
$ geth verkle verify
INFO [날짜] Rebuilding the tree root=0x1234567890abcdef... number=18500000
INFO [날짜] Checking internal node children=256
INFO [날짜] Checking leaf node values=1024
INFO [날짜] Tree was rebuilt from the database


# 특정 루트의 Verkle Tree 검증
```

```
$ geth verkle verify 0x1234567890abcdef1234567890abcdef1234567890abcdef12345

INFO [날짜] Rebuilding the tree root=0x1234567890abcdef...

INFO [날짜] Tree was rebuilt from the database
```

2. `geth verkle dump` - Verkle Tree DOT 파일 덤프

```
{

Name: "dump",

Usage: "Dump a verkle tree to a DOT file",

ArgsUsage: "<root> <key1> [<key 2> ...]",

Action: expandVerkle,

Flags: slices.Concat(utils.NetworkFlags, utils.DatabaseFlags),

Description: `

geth verkle dump <state-root> <key 1> [<key 2> ...]

This command will produce a dot file representing the tree, rooted at <root>.

in which key1, key2, ... are expanded.

`,

}


// expandVerkle 함수

func expandVerkle(ctx *cli.Context) error {

// 1. 노드 스택 생성

stack, _ := makeConfigNode(ctx)
```

```
defer stack.Close()
// 2. 체인 데이터베이스 열기
chaindb := utils.MakeChainDatabase(ctx, stack, true)
defer chaindb.Close()
// 3. 인수 검증 및 파싱
if ctx.NArg() < 2 {
return fmt.Errorf("usage: %s root key1 [key 2...]", ctx.App.Name)
}
var (
rootC common.Hash
keylist [][]byte
err error
)
// 4. 루트 해시 파싱
rootC, err = parseRoot(ctx.Args().First())
if err != nil {
log.Error("Failed to resolve state root", "error", err)
return err
}
// 5. 키 목록 파싱
keylist = make([][]byte, 0, ctx.Args().Len()-1)
args := ctx.Args().Slice()
for i := range args[1:] {
```

```
key, err := hex.DecodeString(args[i+1])
log.Info("decoded key", "arg", args[i+1], "key", key)
if err != nil {
return fmt.Errorf("error decoding key #%d: %w", i+1, err)
}
keylist = append(keylist, key)
}
log.Info("Rebuilding the tree", "root", rootC)
// 6. 직렬화된 루트 노드 가져오기
serializedRoot, err := chaindb.Get(rootC[:])
if err != nil {
return fmt.Errorf("failed to get root from database: %v", err)
}
// 7. Verkle 노드 파싱
root, err := verkle.ParseNode(serializedRoot, 0)
if err != nil {
return fmt.Errorf("failed to parse verkle node: %v", err)
}
// 8. DOT 파일 생성
dotContent, err := generateDOTFile(root, keylist, chaindb.Get)
if err != nil {
return fmt.Errorf("failed to generate DOT file: %v", err)
}
```

```
// 9. 파일 출력
filename := fmt.Sprintf("verkle_tree_%s.dot", rootC.Hex()[:8])
if err := os.WriteFile(filename, []byte(dotContent), 0644); err != nil {
return fmt.Errorf("failed to write DOT file: %v", err)
}
log.Info("DOT file generated", "filename", filename, "keys", len(keylist))
return nil
}


// generateDOTFile는 Verkle Tree를 DOT 형식으로 변환합니다.
func generateDOTFile(root verkle.VerkleNode, keys [][]byte, resolver verkle.Nod
var dot strings.Builder
// 1. DOT 파일 헤더
dot.WriteString("digraph VerkleTree {\\n")
dot.WriteString(" rankdir=TB;\\n")
dot.WriteString(" node [shape=box];\\n\\n")
// 2. 노드 ID 카운터
nodeID := 0
nodeMap := make(map[string]int)
// 3. 재귀적으로 트리 순회
if err := traverseForDOT(root, keys, resolver, &dot, &nodeID, nodeMap, "root");
return "", err
}
```

```
// 4. DOT 파일 푸터
dot.WriteString("}\\n")
return dot.String(), nil
}


// traverseForDOT는 DOT 생성을 위해 트리를 순회합니다.
func traverseForDOT(node verkle.VerkleNode, keys [][]byte, resolver verkle.Node
dot *strings.Builder, nodeID *int, nodeMap map[string]int, label string) error
currentID := *nodeID
*nodeID++
switch n := node.(type) {
case *verkle.InternalNode:
// 1. 내부 노드 표현
dot.WriteString(fmt.Sprintf(" node%d [label=\\\"%s\\\\nInternal\\\", style=fille
currentID, label))
// 2. 자식 노드들 처리
for i, child := range n.Children() {
childC := child.Commit().Bytes()
if bytes.Equal(childC[:], zero[:]) {
continue // 빈 자식 건너뛰기
}
// 3. 이 경로가 관심 키들 중 하나에 해당하는지 확인
shouldExpand := false
```

```
for _, key := range keys {
if len(key) > 0 && int(key[0]) == i {
shouldExpand = true
break
}
}
if shouldExpand {
// 4. 자식 노드 로드 및 재귀 처리
childS, err := resolver(childC[:])
if err != nil {
return fmt.Errorf("could not find child %x: %w", childC, err)
}
childN, err := verkle.ParseNode(childS, 0)
if err != nil {
return fmt.Errorf("decode error child %x: %w", childC, err)
}
childLabel := fmt.Sprintf("child_%d", i)
// 5. 재귀 호출
if err := traverseForDOT(childN, getChildKeys(keys, i), resolver,
dot, nodeID, nodeMap, childLabel); err != nil {
return err
}
// 6. 엣지 추가
```

```
dot.WriteString(fmt.Sprintf(" node%d -> node%d [label=\\\"%d\\\"];\\n",
currentID, *nodeID-1, i))
}
}
case *verkle.LeafNode:
// 7. 리프 노드 표현
dot.WriteString(fmt.Sprintf(" node%d [label=\\\"%s\\\\nLeaf\\\", style=filled, f
currentID, label))
// 8. 리프 노드의 값들 표시
for  i := 0;  i < verkle.NodeWidth;  i++ {
value := n.Value(i)
if  len(value) > 0 {
dot.WriteString(fmt.Sprintf(" node%d_val%d [label=\\\"Value[%d]\\\\n%x\\\", shap
currentID, i, i, value[:min(len(value), 8)]))
dot.WriteString(fmt.Sprintf(" node%d -> node%d_val%d;\\n", currentID, currentID,
}
}
case verkle.Empty:
// 9. 빈 노드 표현
dot.WriteString(fmt.Sprintf(" node%d [label=\\\"%s\\\\nEmpty\\\", style=filled,
currentID, label))
}
return  nil
```

```
}

// getChildKeys는 특정 자식에 해당하는 키들을 필터링합니다.
func getChildKeys(keys [][]byte, childIndex int) [][]byte {
var childKeys [][]byte
for _, key := range keys {
if len(key) > 0 && int(key[0]) == childIndex {
if len(key) > 1 {
childKeys = append(childKeys, key[1:])
} else {
childKeys = append(childKeys, []byte{})
}
}
}
return childKeys
}
```

사용 예시:

```
# 특정 키들을 포함한 Verkle Tree 덤프
$ geth verkle dump 0x1234567890abcdef1234567890abcdef1234567890abcdef1234567
INFO [날짜] decoded key arg=0x742d35cc6634c0532925a3b8d4c9db96c4b4d8b key=[116
INFO [날짜] decoded key arg=0x8ba1f109551bd432803012645bd132fdc6b84b key=[139 1
```

```
INFO [날짜] Rebuilding the tree root=0x1234567890abcdef...

INFO [날짜] DOT file generated filename=verkle_tree_12345678.dot  keys=2



# 생성된 DOT 파일을 Graphviz로 시각화

$  dot  -Tpng  verkle_tree_12345678.dot  -o  verkle_tree.png

$  dot  -Tsvg  verkle_tree_12345678.dot  -o  verkle_tree.svg
```

생성되는 DOT 파일 예시:

```
digraph VerkleTree {

rankdir=TB;

node [shape=box];



node0 [label="root\\nInternal", style=filled, fillcolor=lightblue];

node1 [label="child_116\\nInternal", style=filled, fillcolor=lightblue];

node0 -> node1 [label="116"];

node2 [label="child_45\\nLeaf", style=filled, fillcolor=lightgreen];

node1 -> node2 [label="45"];

node2_val0 [label="Value[0]\\n01000000", shape=ellipse];

node2 -> node2_val0;

node2_val1 [label="Value[1]\\n2a000000", shape=ellipse];

node2 -> node2_val1;

}
```

📊 실행 예시

1. Verkle Tree 검증

```
$ geth verkle verify
INFO [날짜] Rebuilding the tree root=0x1234567890abcdef... number=18500000
INFO [날짜] Checking internal node at depth 0, children=256
INFO [날짜] Checking internal node at depth 1, children=128
INFO [날짜] Checking leaf nodes, count=1024
INFO [날짜] Tree was rebuilt from the database
✅ Verkle tree verification completed successfully
```

2. 특정 루트 검증 (오류 케이스)

```
$ geth verkle verify 0xdeadbeefdeadbeefdeadbeefdeadbeefdeadbeefdeadbeefdeadb
INFO [날짜] Rebuilding the tree root=0xdeadbeefdeadbeef...
ERROR [날짜] Could not rebuild the tree from the database err="could not find chi
❌ Verkle tree verification failed
```

3. DOT 파일 생성

```
$ geth verkle dump 0x1234567890abcdef1234567890abcdef1234567890abcdef1234567
INFO [날짜] decoded key arg=0x742d35cc6634c0532925a3b8d4c9db96c4b4d8b key=[116
```

```
INFO [날짜] Rebuilding the tree root=0x1234567890abcdef...

INFO [날짜] Generating DOT representation...

INFO [날짜] DOT file generated filename=verkle_tree_12345678.dot  keys=1



# Graphviz로 시각화

$  dot  -Tpng  verkle_tree_12345678.dot  -o  verkle_tree.png

$  open  verkle_tree.png  # macOS

$  xdg-open  verkle_tree.png  # Linux
```

🔧 Verkle Tree 구조 이해

노드 타입들

```
// Verkle Tree의 노드 타입들

type VerkleNode interface {

Commit() *verkle.Point

// ... 기타 메서드들

}



// 1. 내부 노드 (Internal Node)

type InternalNode struct {

children [NodeWidth]VerkleNode // 256개의 자식

// ...
```

```
}


// 2. 리프 노드 (Leaf Node)
type LeafNode struct {
values [NodeWidth][]byte  // 256개의 값
// ...
}


// 3. 빈 노드 (Empty Node)
type Empty struct{}
```

키-값 매핑

```
Ethereum 계정 주소: 0x742d35Cc6634C0532925a3b8D4C9db96c4b4d8b
↓ Keccak256 해시
계정 해시: 0x1234567890abcdef1234567890abcdef1234567890abcdef1234567890abcdef
↓ Verkle Tree 키 변환
Verkle 키: [0x12, 0x34, 0x56, 0x78, ...]
↓ 트리 경로
트리 경로: root → child[0x12] → child[0x34] → ... → leaf
```

값 저장 구조

```
리프 노드의 각 슬롯:
├── Slot 0: Balance (32 bytes)
├── Slot 1: Nonce (32 bytes)
├── Slot 2: Code Hash (32 bytes)
├── Slot 3: Storage Root (32 bytes)
└── Slot 4-255: Storage Values
```

🛠️ 문제 해결

일반적인 오류들

1. Verkle 지원 없음

```
Fatal: verkle tree not supported in this build
```

해결: Verkle 지원이 활성화된 geth 빌드 사용

2. 노드 파싱 오류

```
ERROR [날짜] decode error child abc123 in db: invalid node format
```

해결: 올바른 Verkle 형식의 데이터베이스 확인

3. 키 형식 오류

```
error decoding key #1: encoding/hex: invalid byte: U+0047 'G'
```

해결: 16진수 형식의 키 사용 (0x 접두사 제외)

DOT 파일 시각화

Graphviz 설치:

```
# Ubuntu/Debian
sudo apt-get install graphviz

# macOS
brew install graphviz

# Windows
# https://graphviz.org/download/ 에서 다운로드
```

다양한 출력 형식:

```
# PNG 이미지
dot -Tpng verkle_tree.dot -o verkle_tree.png

# SVG 벡터
dot -Tsvg verkle_tree.dot -o verkle_tree.svg

# PDF 문서
dot -Tpdf verkle_tree.dot -o verkle_tree.pdf

# 대화형 HTML
```

```
dot -Tsvg verkle_tree.dot | sed 's/svg/svg onload="init(evt)"/' > interactive
```

🔮 Verkle Tree의 미래

로드맵

1. **Phase 1**: 실험적 구현 (현재)
2. **Phase 2**: 테스트넷 배포
3. **Phase 3**: 메인넷 하드포크
4. **Phase 4**: MPT 완전 대체

예상 이점

```
현재 (MPT) 미래 (Verkle)
├── 상태 증명: ~1KB ├── 상태 증명: ~150B
├── 동기화: 수 시간 ├── 동기화: 수 분
├── 스토리지: ~500GB ├── 스토리지: ~50GB
└── Stateless: 불가능 └── Stateless: 가능
```

개발자 준비사항

1. **Verkle 이해**: 새로운 트리 구조 학습
2. **도구 업데이트**: Verkle 지원 도구 개발
3. **테스트**: 테스트넷에서 미리 검증
4. **마이그레이션**: MPT → Verkle 전환 계획

📈 고급 사용법

1. 자동화된 검증 스크립트

```
#!/bin/bash
# verkle_verify.sh

DATADIR="/path/to/ethereum/data"
LOGFILE="/var/log/verkle_verify.log"

echo "$(date): Starting Verkle verification" >> $LOGFILE

# 최근 100개 블록의 Verkle Tree 검증
for i in {0..99}; do
BLOCK_NUM=$(($(geth --datadir $DATADIR --exec "eth.blockNumber" attach) - i
ROOT=$(geth --datadir $DATADIR --exec "eth.getBlock($BLOCK_NUM).stateRoot"
echo "Verifying block $BLOCK_NUM (root: $ROOT)" >> $LOGFILE
if geth --datadir $DATADIR verkle verify $ROOT >> $LOGFILE 2>&1; then
echo "✅ Block $BLOCK_NUM verified" >> $LOGFILE
else
echo "❌ Block $BLOCK_NUM failed verification" >> $LOGFILE
exit 1
fi
```

```
done

echo "$(date): All verifications completed successfully" >> $LOGFILE
```

2. 트리 비교 도구

```
#!/bin/bash
# compare_trees.sh

ROOT1=$1
ROOT2=$2

if [ -z "$ROOT1" ] || [ -z "$ROOT2" ]; then
echo "Usage: $0 <root1> <root2>"
exit 1
fi

echo "Comparing Verkle trees:"
echo "Root 1: $ROOT1"
echo "Root 2: $ROOT2"

# 공통 키들로 두 트리 덤프
KEYS="742d35cc6634c0532925a3b8d4c9db96c4b4d8b 8ba1f109551bd432803012645bd132fdc6
```

```
geth verkle dump $ROOT1 $KEYS
mv verkle_tree_*.dot tree1.dot

geth verkle dump $ROOT2 $KEYS
mv verkle_tree_*.dot tree2.dot

# 차이점 비교
diff tree1.dot tree2.dot > tree_diff.txt

if [ -s tree_diff.txt ]; then
echo "❌ Trees are different. See tree_diff.txt for details."
else
echo "✅ Trees are identical."
fi
```

참고 자료:

- Verkle Trees EIP
- Verkle Tree 논문
- Ethereum Verkle 로드맵
- go-verkle 라이브러리

이 문서는 go-ethereum 소스 코드 분석을 바탕으로 작성되었습니다.